Demand Planning Analyst: Project Report

Oct 21, 2026 · @student

We built a working, locally hosted AI demand-planning analyst for the beauty & personal-care dataset: Claude plans the analysis, and deterministic Python tools compute every number. It passes 32 automated tests and the full end-to-end flow; the remaining step before the Oct 26, 2026 demo is one live run with the real Claude API on the presenting laptop.

Status

All six planned phases are built and tested offline; the only untested piece is the live connection to Claude, because the build workspace could not download packages.

Phase	What it delivers	Status
(a) Upload, profiling, joins, quality	ZIP/CSV/Excel loading with guardrails, key and relationship inference, join validation, data quality report	Built, 17 tests pass, real dataset loads
(b) Chat and agent loop	Chat UI, Claude tool-use loop, descriptive analytics, interactive charts, audit trail	Built; tested with a scripted stand-in for Claude
(c) Weekly forecasting	3 baselines + ETS + LightGBM, rolling-origin backtest, prediction intervals	Built; backtested on the real data
(d) Scenario and impact analysis	Draft-then-confirm scenarios, landed cost and margin, sensitivity, mitigations, source badges	Built; South Korea and India tariffs tested
(e) Session Reference sidebar	Editable data dictionary, relationships, KPI glossary with worked examples, assumptions log, quality badges	Built

Phase	What it delivers	Status
(f) Next questions, polish, docs	Answer-specific question chips, error handling, README, DEMO_SCRIPT, end-to-end test	Built; live dry run pending

The project lives in `Documents\demand-planning-agent` on the presenting laptop and starts with one command: `uv run python start.py`.

The dataset

The ZIP holds 23 tables in 29 files; it is unusually clean, and two findings changed how the scenario logic had to work.

Area	Tables	Grain	Size
Sales	fact_weekly_sales	SKU × DC × channel × week	111,001 rows, 581 series
Inventory	fact_weekly_inventory	SKU × DC × week (no channel)	56,302 rows
Purchasing	fact_purchase_orders, product_supplier, dim_supplier	PO line	20,130 POs, 12 suppliers
Planning inputs	dim_promotion, promotion_scope, fact_price_list, fact_marketing_weekly, assortment_policy	various	194 promotions, 243 price changes
External factors	weather, climate normals, macro, competitor price, social signal, 22 events + scope	various	
Dimensions	product (60 SKUs, 3 categories, 5 brands), DC (5), channel (2), calendar (260 weeks)		

- **History:** Monday 2022-10-03 to 2026-09-21 (208 weeks), plus 52 future weeks of calendar and planned marketing.

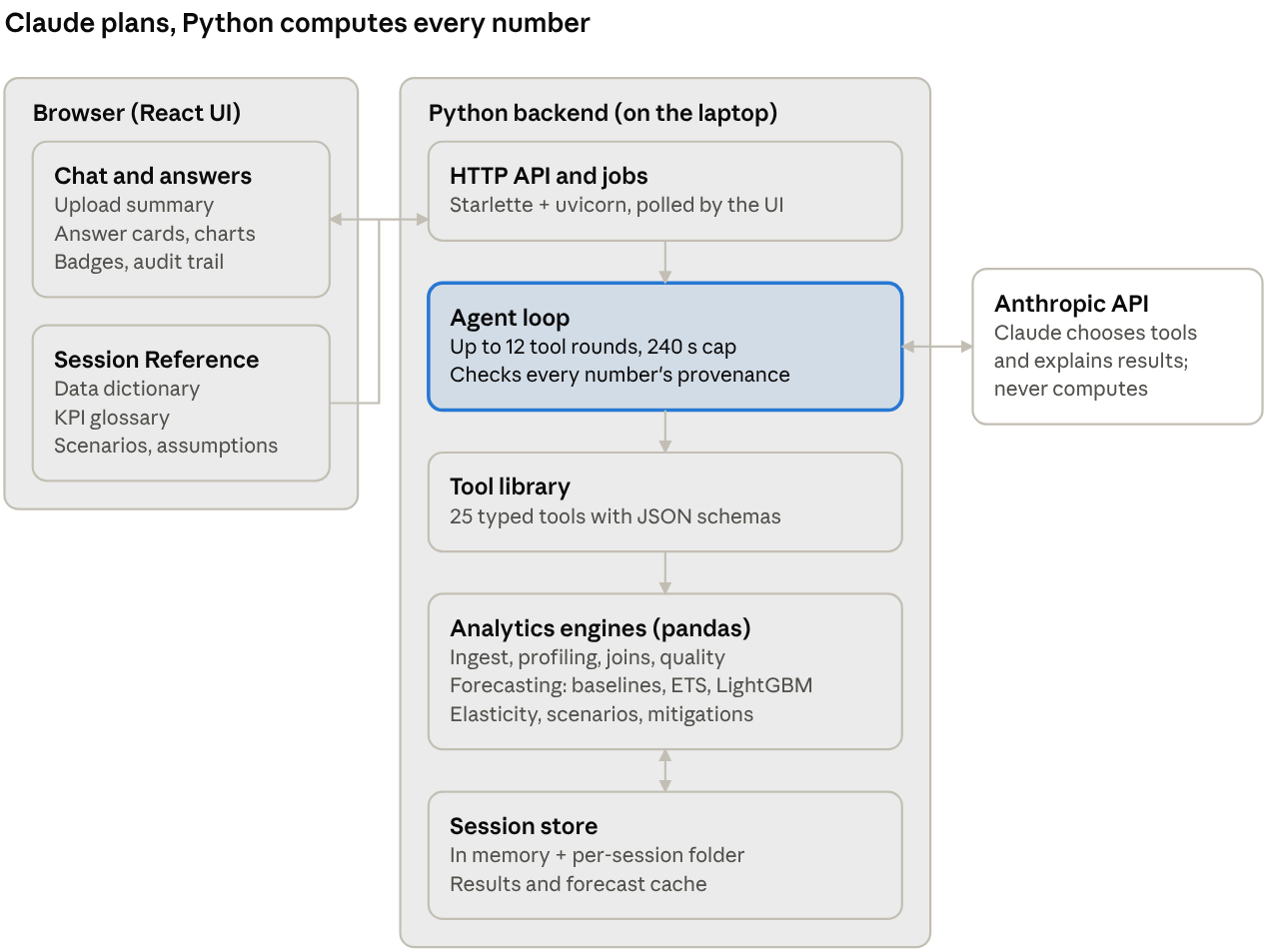
- **Keys:** all 23 primary keys are unique and all 45 inferred relationships match 100% of keys, including composite ones such as (SKU, supplier) on purchase orders.
- **Six tables arrive as both CSV and Excel.** Their contents are identical; the app keeps the CSV and reports the duplicate.
- **Joins that need care:** inventory has no channel, so joining it straight onto sales double-counts stock (the app warns and aggregates first). Competitor price joins on category and macro data on month; both are flagged "aggregate first".
- **Quality:** no missing weeks; 2,657 zero-demand rows; 25 series under a year of history; stockouts are recorded explicitly (4.0% of rows, 2.48% of demand lost), so we forecast unconstrained `customer_demand`.

Finding 1: there is no Indian supplier. Suppliers are in South Korea, France, Italy, China, Mexico and the US, so the brief's India tariff example correctly shows zero exposure. We demo it as the honest no-exposure case and use South Korea for the full impact story.

Finding 2: demand response to a permanent price rise cannot be estimated. Promotions give a clear elasticity of about −3, but list-price changes alone produce intervals that cross zero. Applying −3 to tariff pass-through would overstate the volume loss, so the app shows a hypothetical sensitivity table and asks instead.

System architecture

A question flows from the browser to a Python backend on the same laptop; Claude only decides which tools to call and writes the explanation, while every number comes from the analytics engines.



system architecture · browser, Python backend, Anthropic API

One turn: the UI posts the message and polls a job; the agent sends Claude the rules, live session context and tool list; Claude calls tools until it delivers a structured final answer; the backend checks every number against the tool results before the answer and its charts reach the screen.

Choice	Why
Python analytics (pandas, statsmodels, LightGBM, scipy)	Standard, testable and defensible; all maths is deterministic
Starlette + uvicorn (the core FastAPI is built on)	About 15 JSON routes need nothing more; available offline during the build
Jobs + polling instead of websockets	Robust on classroom networks; shows each step as it runs

Choice	Why
In-memory store, no database	About 110k rows fit in memory; every join still goes through validated row-count and fan-out checks
React + TypeScript bundled with esbuild	One prebuilt bundle, so the laptop needs no Node.js
Plotly served from the Python package	Charts work without internet access

How the analytics work

The model never computes: it chooses from 25 typed tools, and each tool is plain, unit-tested Python.

Area	Tools
Data	list_tables, profile_tables, infer_relationships, confirm_relationships, join_tables, data_quality_report
Descriptive	aggregate (auto-joins dimension attributes), descriptive_stats, make_chart, detect_outliers, calculate
Forecasting	forecast_weekly, backtest, compare_models, forecast_chart
Scenarios	estimate_price_elasticity, create_scenario, confirm_scenario, apply_scenario, compare_scenarios, sensitivity_analysis, supplier_cost_impact, financial_impact, evaluate_mitigations
Output	final_answer (the 8-part response: answer, key results, chart, method, data notes, assumptions, recommendations, next questions)

Forecasting.

- Any additive weekly measure (demand, revenue, units sold, PO quantity) at total, category, SKU or any dimension level.
- Baselines always run: naive, seasonal naive (52-week lag) and an 8-week moving average.
- Candidates: ETS/Holt-Winters (statsmodels) and LightGBM, one global model using a direct multi-horizon strategy so errors do not compound after peaks such as Black Friday.
- Validation: rolling-origin backtest inside the last 52 weeks (4 origins, 13-week horizon), reporting WAPE, MAE, RMSE, bias and MASE; MAPE only with its zero-week caveat.
- A model is recommended only if it beats the best baseline; prediction intervals come from the backtest's own errors.

Scenarios.

1. The event is parsed into a structured draft: type, scope, dates, rates, pass-through, demand effect (default: none). The user must confirm it; the tool refuses to apply drafts.
2. Direct effects first: landed cost = PO unit cost × (1 + tariff) + freight per unit, blended by sourcing share, with stock already bought protected at the old cost (FIFO).
3. Demand effects only from a usable estimated elasticity or a user-stated value; otherwise a sensitivity table labelled hypothetical.
4. Every answer covers normal conditions, what changes, what is affected (ranked by exposure) and mitigations: shift volume (capacity, lead time, MOQ), pre-buy (lead-time feasibility, air expedite), re-price and safety stock.
5. Every number carries a badge: **Computed** (data + stated parameters), **Estimated** (depends on a model) or **Assumed** (user or hypothetical).

Data quality. Outliers use a robust z-score on seasonally adjusted log demand ($|z| > 3.5$); they are never deleted and each is explained by promotions, holidays or in-scope events.

Results so far

The ML model beats the best baseline at every level, and the South Korea tariff costs about $207k of gross margin over 26 weeks before any mitigation.

Level	Seasonal naive WAPE (best baseline)	ETS WAPE	ML model WAPE
Total	5.3%	7.3%	3.4%

Level	Seasonal naive WAPE (best baseline)	ETS WAPE	ML model WAPE
Category	8.7%	9.7%	6.8%
SKU	23.7%	24.3%	15.6%

WAPE = total forecast error as a share of actual demand; lower is better. These backtests ran with scikit-learn's gradient boosting as the stand-in for LightGBM and must be re-run once LightGBM is installed.

South Korea +25% tariff from 2026-10-29 (10 SKUs, suppliers SUP01 and SUP02):

- Landed cost rises $0.6 to $2.9 per unit (Computed); Vitamin C Serum (SKU40) is the most exposed SKU.
- Gross margin falls about $207k over 26 weeks with no demand effect applied, about $620k a year at full run rate (Estimated).
- About 40k units of stock on hand or arriving before the start date are protected at the old cost.
- Pre-buying is not feasible: Korean suppliers need 8 weeks against 9 days to the start date, and historical air freight is too slow.
- Shifting volume to already-contracted US and French suppliers saves about $46k within the horizon; 7 SKUs have no alternate supplier and are flagged as not evaluable.

The India +25% tariff returns 0 SKUs exposed, with the list of countries the company actually imports from.

Human values

The product is designed so a planner can trust, check and overrule every number; these principles are built into the code, not just the prompt.

Value	What it means here	How the app enforces it
Honesty	No invented numbers or confident guesses	Tools compute everything; a provenance check flags any number in an answer not found in a tool result, user input or logged assumption

Value	What it means here	How the app enforces it
Transparency	Anyone can see how an answer was produced	"How this was computed" lists every tool call with its parameters and results; the KPI glossary shows formulas and worked examples
Human control	People make the consequential decisions	Relationships and scenarios start as drafts; `apply_scenario` refuses until the user confirms; demand assumptions are asked for, never defaulted silently
Humility	Uncertainty is shown, not hidden	Prediction intervals on every forecast; Computed / Estimated / Assumed badges; models recommended only if they beat simple baselines
Integrity of data	Nothing is quietly cleaned away	Outliers are flagged, never deleted; duplicates, skipped files and join fan-out are reported
Privacy and security	Company data stays under the company's control	Runs locally; the API key lives in `.env`, never in code; full tables stay on the laptop, but tool results (aggregates, profiles, up to 60 sample rows) are sent to the Anthropic API
Accessibility	Usable by planners with or without data-science training	Plain-language answers, jargon defined on first use, source badges that use shape as well as colour

Where people must stay in the loop:

- Confirming table relationships before analyses build on them.
- Confirming every scenario's parameters and default assumptions before it is applied.
- Choosing a demand-response assumption when the data cannot estimate one.
- Judging recommendations against context the data does not hold: supplier relationships, contracts, brand strategy, customer commitments.

- Owning the final decision. The app is decision support; a planner signs off on any order, price change or supplier shift.

Known limits to say out loud: the provenance check catches invented numbers, not flawed reasoning; elasticities are averages across SKUs; tariff mechanics are simplified (rate on unit cost, freight untaxed); only already-contracted suppliers are considered for volume shifts.

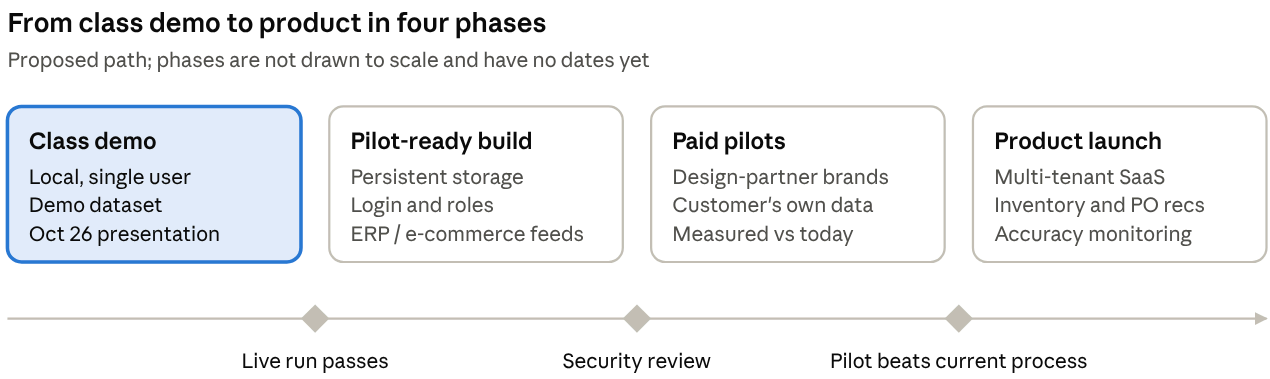
Before the demo

One dry run on the presenting laptop closes every open item; the exact prompts are in `DEMO_SCRIPT.md`.

- [ ] Install uv and run `uv run python start.py`; confirm the header shows the model name, not "AI not connected"
- [ ] Add a funded Anthropic API key to `.env` (the API is billed separately from Claude subscriptions)
- [ ] Load demo data and time the upload (target under 15 seconds with the fast Excel reader)
- [ ] Confirm charts render (plotly.js is served locally from the Python package)
- [ ] Run `uv run pytest` and confirm all 32 tests pass on Windows
- [ ] Re-run the forecast and note the WAPE figures with real LightGBM for the slides
- [ ] Walk the demo script end to end with the live model and screenshot each step as a fallback
- [ ] Rehearse the scripted backup mode in case the classroom network blocks the API
- [ ] Send any errors (browser + PowerShell screenshots) to Claude for fixes

Future scope: taking it to customers

The pitch is an analyst that explains itself: a mid-sized brand describes an event in plain English and gets a quantified, auditable impact in minutes. Turning the class project into a product needs persistence, security and live data connections before any customer pilot.



proposed roadmap · 4 phases, 3 gates

Each gate must be passed before the next phase starts: a working live demo, a security review customers will accept, then pilots that beat the customer's current forecasting process.

Who would buy. Mid-sized consumer brands (beauty, personal care, food, apparel) whose planners live in spreadsheets: too small or too stretched for enterprise planning suites such as SAP IBP, Kinaxis, o9 or Blue Yonder, but exposed to the same tariffs, supplier failures and promotions.

Why they would switch.

- Event-to-impact in minutes instead of a week of spreadsheet work.
- Every number is traceable, which planners need when they defend a plan to finance.
- Forecasts are benchmarked against simple baselines, so customers see whether the model earns its place.
- Mitigations come ranked with constraints (capacity, lead time, MOQ), not just a bigger number.

What must be built before selling.

Capability	Why customers need it	Today
Persistent storage and multi-user sessions	Plans, scenarios and edits must survive a restart and be shared by a team	In-memory, single user
Login, roles, SSO	Only approved people can apply scenarios or see costs	None (local only)

Capability	Why customers need it	Today
Security and compliance (SOC 2, data residency, data-processing terms with the model vendor)	Procurement will not sign without it	Local laptop only
Data connectors (ERP, e-commerce, EDI) and scheduled refresh	Customers will not upload ZIPs every week	File upload only
Scale (DuckDB or a warehouse, background workers)	Real catalogues have thousands of SKUs	About 110k rows comfortable
Richer models (intermittent demand, hierarchy reconciliation, promotion lift)	Long-tail SKUs and promo planning	Baselines, ETS, LightGBM
Inventory and purchase-order recommendations with approval workflow	Planners act on orders, not forecasts	Mitigations are advisory
Accuracy monitoring over time	Trust is earned week by week	Backtest only
LLM cost controls and vendor fallback	Predictable margins	Single model, no budget cap

Business model (proposed). Subscription per planner seat with a data-volume tier, sold after a paid pilot on the customer's own data. The pilot's success test: the forecast beats the customer's current process on WAPE, and scenario turnaround drops from days to hours.

Risks to manage. Customers acting on a wrong recommendation (keep human sign-off and the audit trail); messy real data (the quality report becomes onboarding); per-question model costs; dependence on one model vendor; sending cost and supplier data to a third-party API (offer contractual and technical controls).